그래프를 이용한 기계 학습

#7 그래프의 정점을 어떻게 벡터로 표현할까?

신기정

(KAIST AI대학원)

© NAVER Connect Foundation

1. 정점 표현 학습

2. 인접성 기반 접근법

3. 거리/경로/중첩 기반 접근법

4. 임의보행 기반 접근법

5. 변환식 정점 표현 학습의 한계

© NAVER Connect Foundation

1. 정점 표현 학습

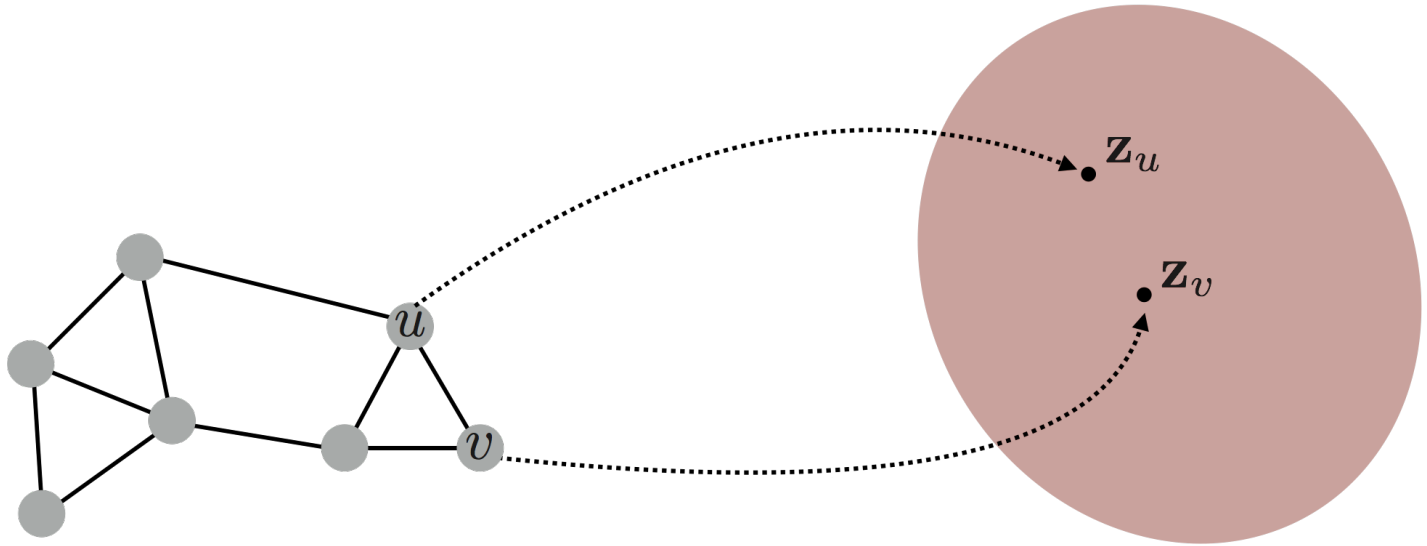
1. 정점 표현 학습이란?
2. 정점 표현 학습의 이유
3. 정점 표현 학습의 목표

© NAVER Connect Foundation

1.1 정점 표현 학습이란?

정점 표현 학습이란 **그래프의 정점들을 벡터의 형태로 표현하는 것**입니다

정점 표현 학습은 간단히 **정점 임베딩(Node Embedding)**이라고도 부릅니다
정점 임베딩은 벡터 형태의 표현 그 자체를 의미하기도 합니다
정점이 표현되는 벡터 공간을 **임베딩 공간**이라고 부릅시다



입력 그래프 임베딩 공간

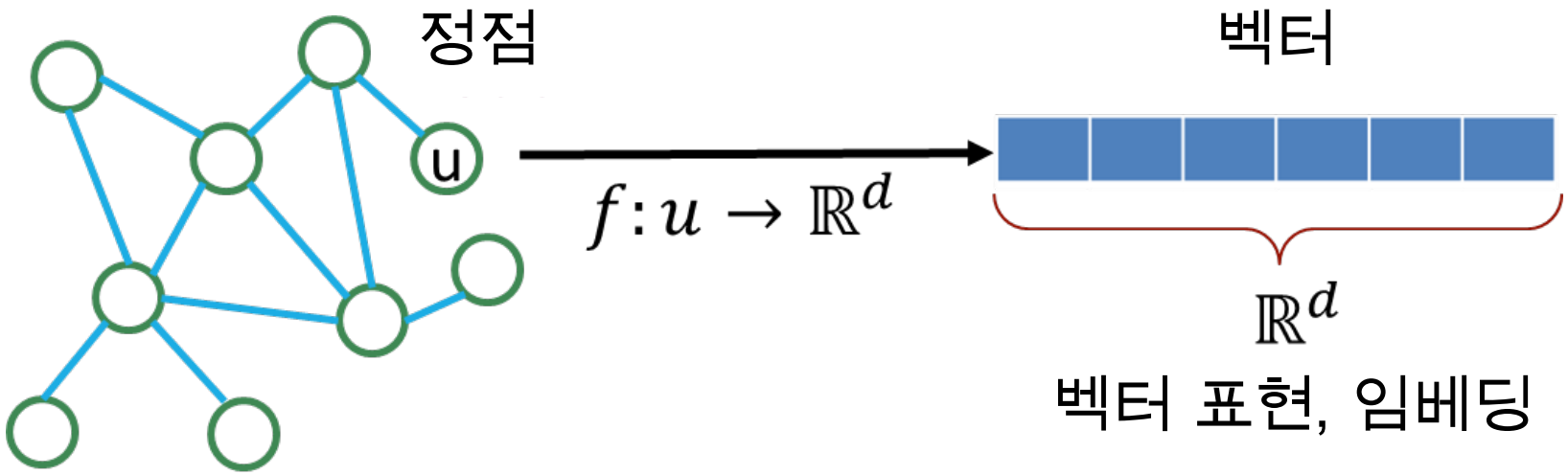
© NAVER Connect Foundation

이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

1.1 정점 표현 학습이란?

정점 표현 학습이란 **그래프의 정점들을 벡터의 형태로 표현하는 것**입니다

정점 표현 학습의 입력은 그래프입니다
주어진 그래프의 각 정점 u에 대한 임베딩, 즉 벡터 표현 $\mathbf{z}_u$가 정점 임베딩의 출력입니다



© NAVER Connect Foundation

이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

1.2 정점 표현 학습의 이유

정점 임베딩의 결과로, 벡터 형태의 데이터를 위한 도구들을 그래프에도 적용할 수 있습니다

기계학습 도구들이 한가지 예시입니다
대부분 분류기(로지스틱 회귀분석, 다층 퍼셉트론 등) 그리고
군집 분석 알고리즘(K-Means, DBSCAN 등)은
벡터 형태로 표현된 사례(Instance)들을 입력으로 받습니다

그래프의 정점들을 벡터 형태로 표현할 수 있다면,
위의 예시와 같은 대표적인 도구들 뿐 아니라, 최신의 기계 학습도구들을
정점 분류(Node Classification), 군집 분석(Community Detection) 등에
활용할 수 있습니다

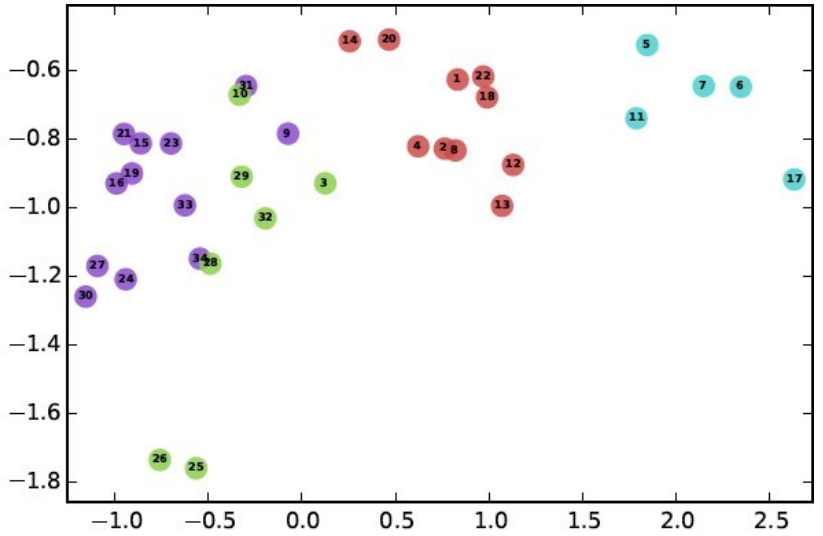
© NAVER Connect Foundation

1.3 정점 표현 학습의 목표

어떤 기준으로 정점을 벡터로 변환해야할까요?

그래프에서의 정점간 유사도를 임베딩 공간에서도 “보존”하는 것을 목표로 합니다





© NAVER Connect Foundation

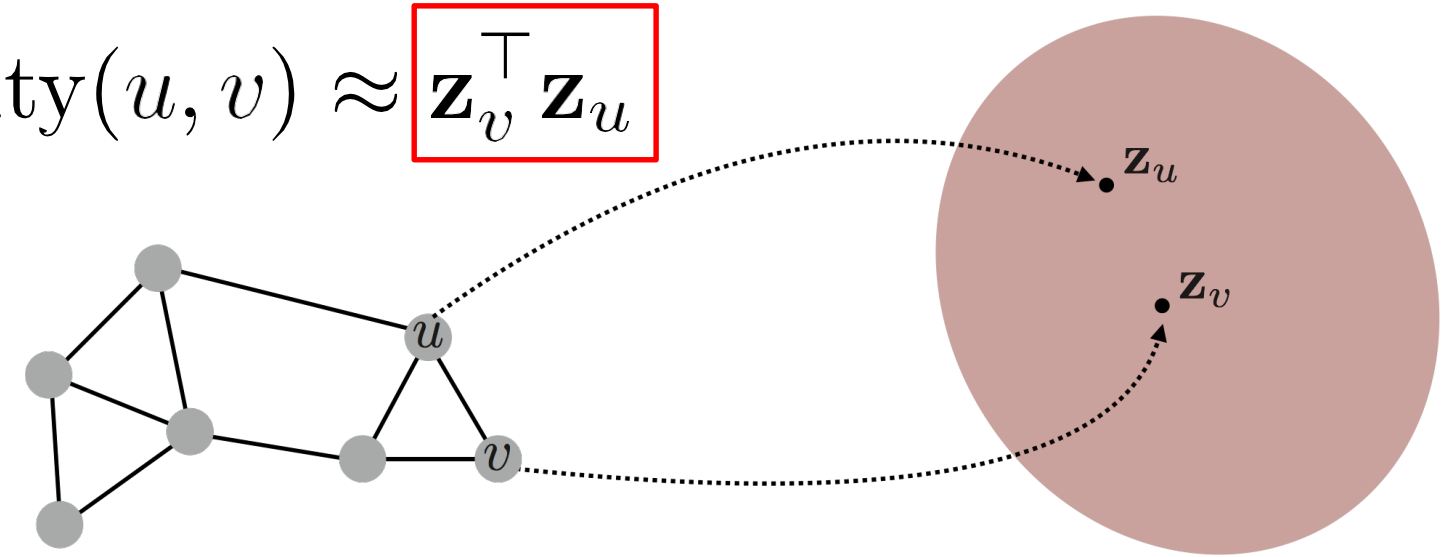
이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

1.3 정점 표현 학습의 목표

임베딩 공간에서의 유사도로는 내적(Inner Product)를 사용합니다

임베딩 공간에서의 u와 v의 유사도는 둘의 임베딩의 내적 $\boldsymbol{z_v^T z_u = ||z_u|| \cdot ||z_v|| \cdot cos(\theta)}$ 입니다
내적은 두 벡터가 클 수록, 그리고 같은 방향을 향할 수록 큰 값을 갖습니다

$$\text{similarity}(u, v) \approx \mathbf{z}_v^\top \mathbf{z}_u$$



입력 그래프 임베딩 공간

© NAVER Connect Foundation

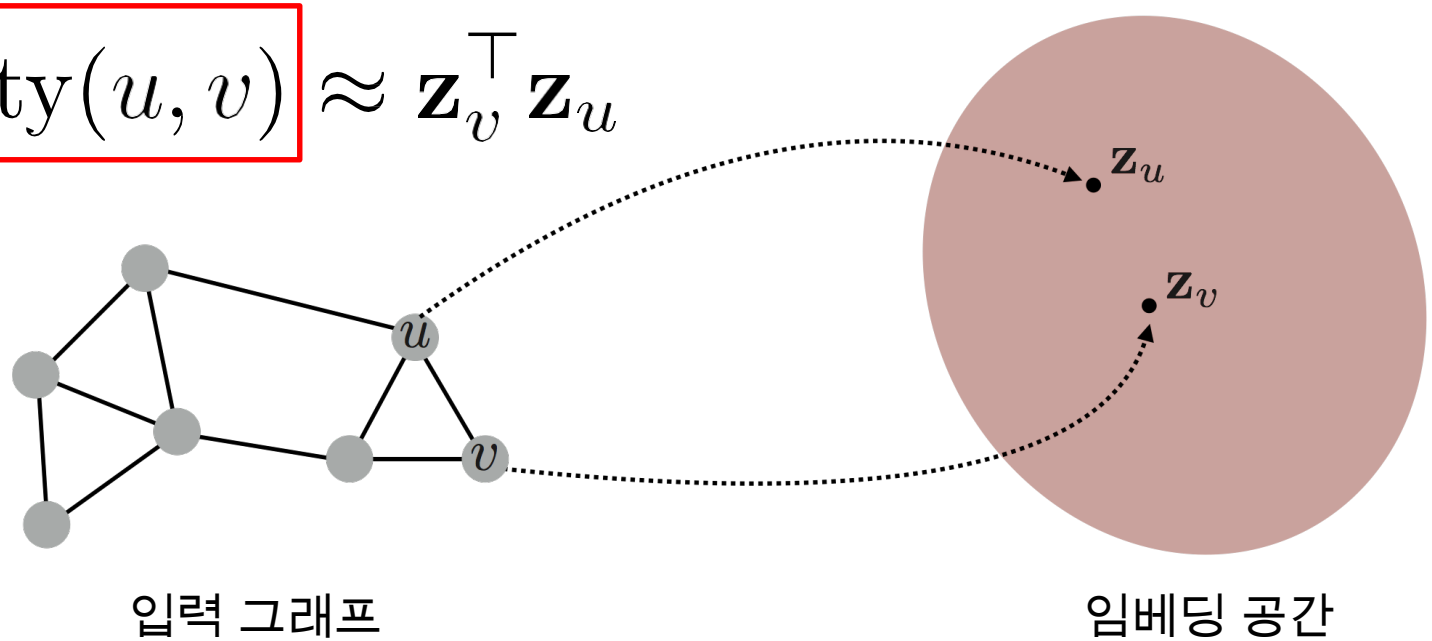
이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

1.3 정점 표현 학습의 목표

그래프에서 두 정점의 유사도는 어떻게 정의할까요?

이 질문에는 여러가지 답이 있을 수 있습니다
본 강의에서는 대표적인 답을 몇 가지를 설명합니다

$$\text{similarity}(u, v) \approx \mathbf{z}_v^\top \mathbf{z}_u$$



© NAVER Connect Foundation

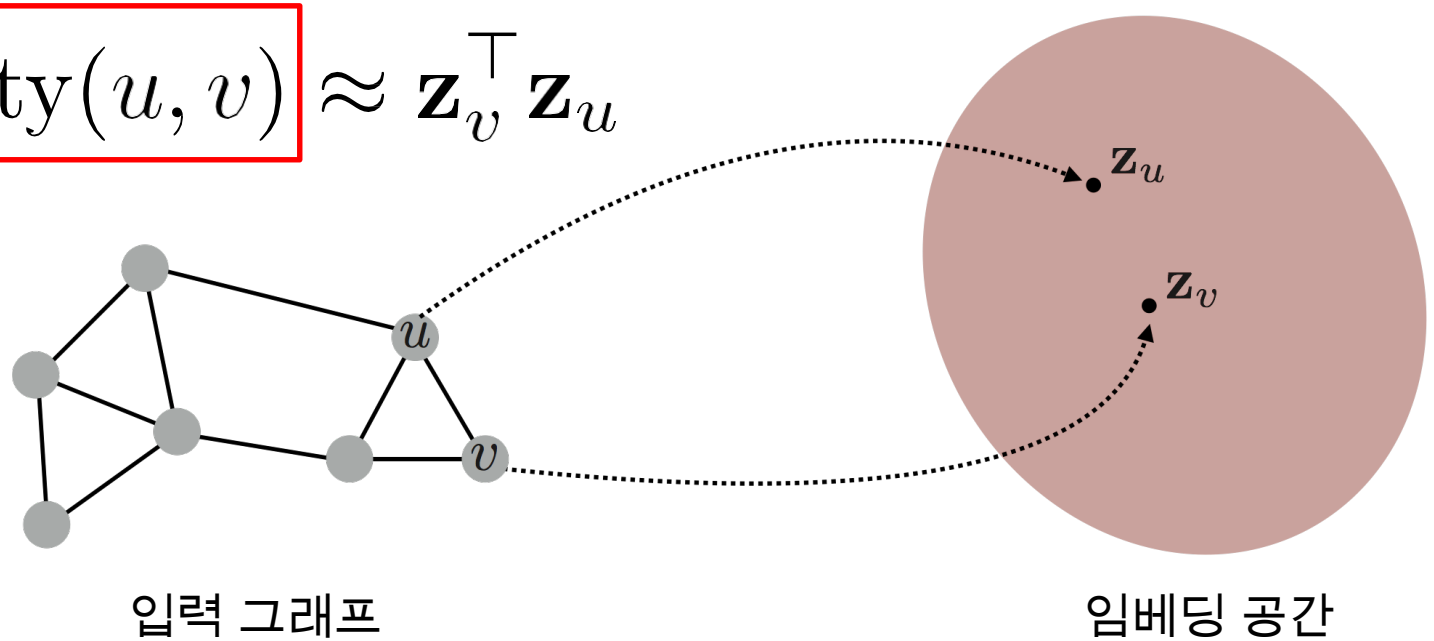
이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

1.3 정점 표현 학습의 목표

정리하면, 정점 임베딩은 다음 두 단계로 이루어집니다

1) 그래프에서의 정점 유사도를 정의하는 단계
2) 정의한 유사도를 보존하도록 정점 임베딩을 학습하는 단계

$$\text{similarity}(u, v) \approx \mathbf{z}_v^\top \mathbf{z}_u$$



입력 그래프 임베딩 공간

© NAVER Connect Foundation

이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

2. 인접성 기반 접근법

그래프에서, 두 정점의 유사도 정의 방법 - 1

1. 인접성 기반 접근법
2. 인접성 기반 접근법의 한계

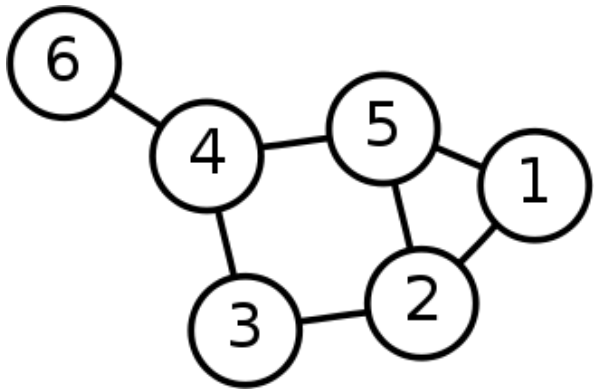
© NAVER Connect Foundation

2.1 인접성 기반 접근법

인접성(Adjacency) 기반 접근법에서는 **두 정점이 인접할 때 유사**하다고 간주합니다

두 정점 u와 v가 인접하다는 것은 둘을 직접 연결하는 간선 (u, v)가 있음을 의미합니다

인접행렬(Adjacency Matrix) $\mathbf{A}$의 u행 v열 원소 $\mathbf{A}_{u,v}$는 u와 v가 인접한 경우 1아닌 경우 0입니다
인접행렬의 원소 $\mathbf{A}_{u,v}$ 를 두 정점 u와 v의 유사도로 가정합니다



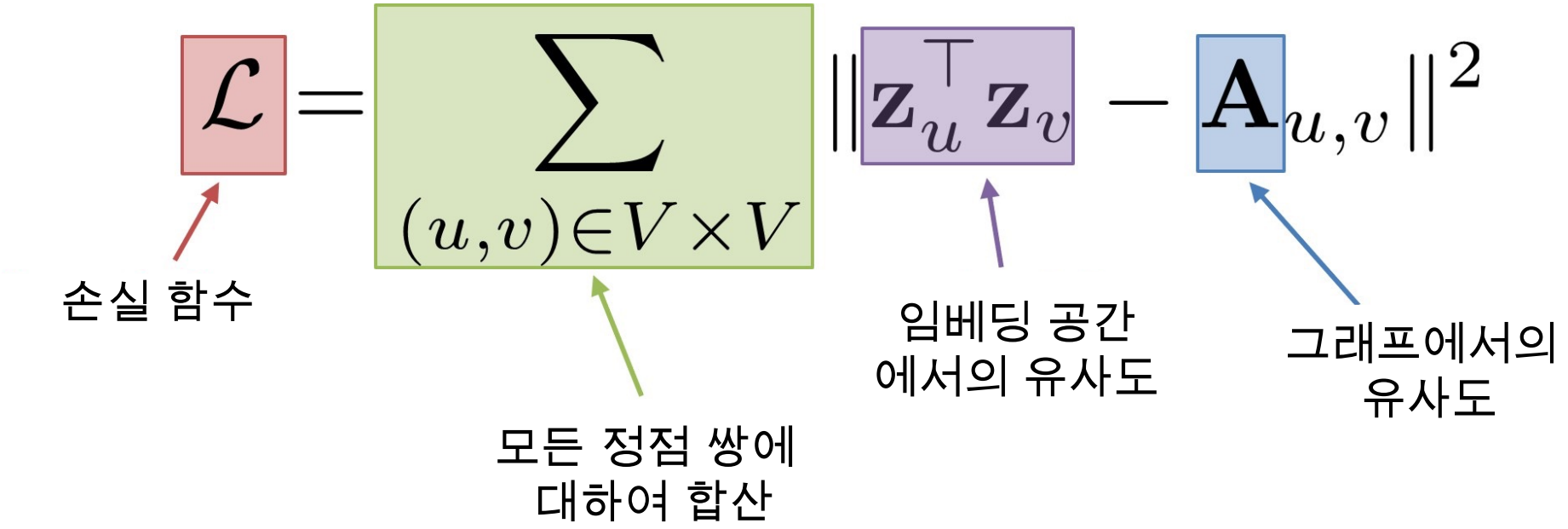
	1	2	3	4	5	6
1	0	1	0	0	1	0
2	1	0	1	0	1	0
3	0	1	0	1	0	0
4	0	0	1	0	1	1
5	1	1	0	1	0	0
6	0	0	0	1	0	0

© NAVER Connect Foundation

2.1 인접성 기반 접근법

인접성 기반 접근법의 손실 함수(Loss Function)는 아래와 같습니다

즉, 이 손실 함수가 최소가 되는 정점 임베딩을 찾는 것을 목표로 합니다
손실 함수 최소화를 위해서는 (확률적) 경사하강법 등이 사용됩니다

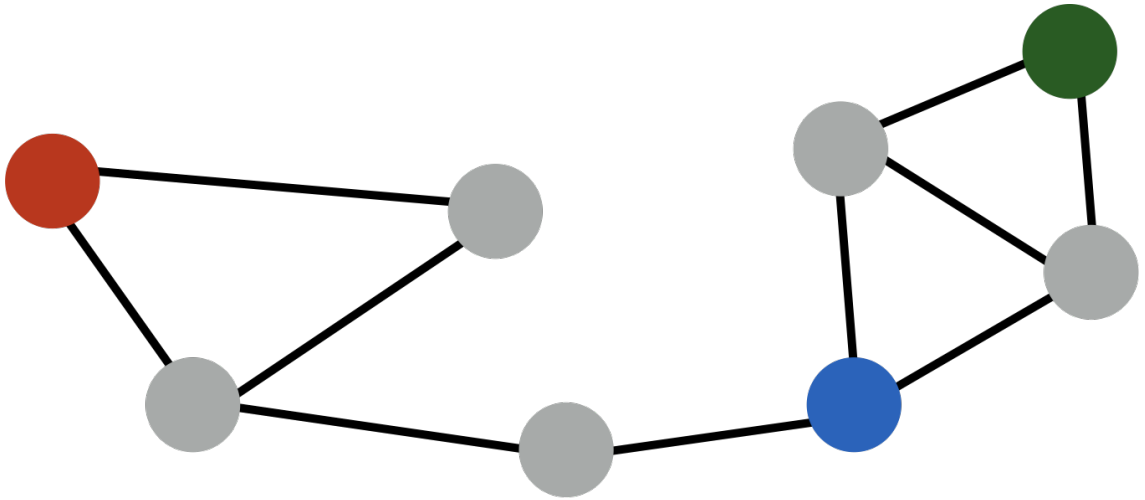


© NAVER Connect Foundation

2.2 인접성 기반 접근법의 한계

인접성만으로 유사도를 판단하는 것은 한계가 있습니다

빨간색 정점과 파란색 정점은 거리가 3인 반면
초록색 정점과 파란색 정점은 거리가 2입니다
인접성만을 고려할 경우 이러한 사실에 대한 고려 없이, 두 경우의 유사도는 0으로 같습니다



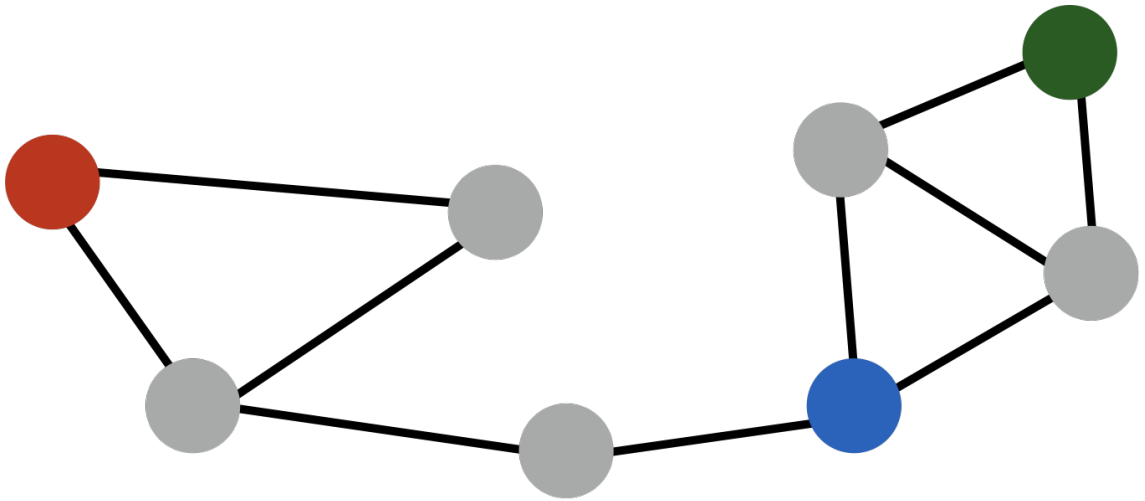
© NAVER Connect Foundation

이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

2.2 인접성 기반 접근법의 한계

인접성만으로 유사도를 판단하는 것은 한계가 있습니다

군집 관점에서는 빨간색 정점과 파란색 정점은 다른 군집에 속하는 반면
초록색 정점과 파란색 정점은 다른 군집에 속합니다
인접성만을 고려할 경우 이러한 사실에 대한 고려 없이, 두 경우의 유사도는 0으로 같습니다



© NAVER Connect Foundation

이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

3. 거리/경로/중첩 기반 접근법

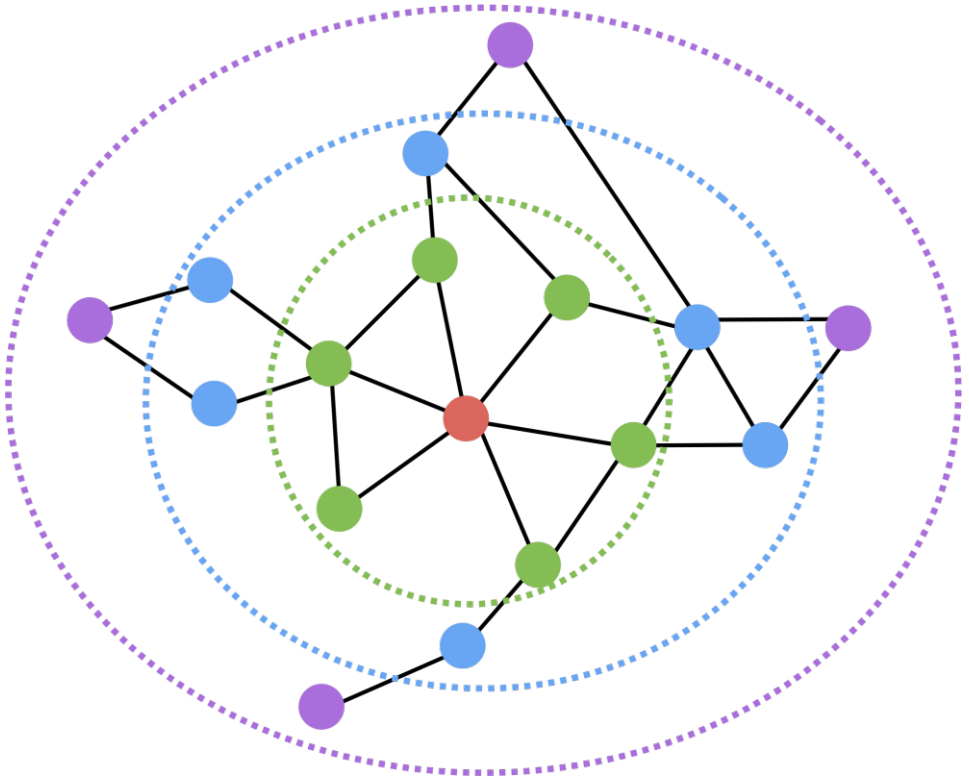
그래프에서, 두 정점의 유사도 정의 방법 - 2

1. 거리 기반 접근법
2. 경로 기반 접근법
3. 중첩 기반 접근법

© NAVER Connect Foundation

3.1 거리 기반 접근법

거리 기반 접근법에서는 **두 정점 사이의 거리가 충분히 가까운 경우** 유사하다고 간주합니다



예를 들어, “충분히”의 기준을 2로 가정합시다

빨간색 정점은 초록색 그리고 파란색 정점들과 유사합니다. 즉 유사도가 1입니다

반면, 빨간색 정점은 보라색 정점과는 유사하지 않습니다. 즉 유사도가 0입니다

© NAVER Connect Foundation

이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

3.2 경로 기반 접근법

경로 기반 접근법에서는 두 정점 사이의 경로가 많을 수록 유사하다고 간주합니다

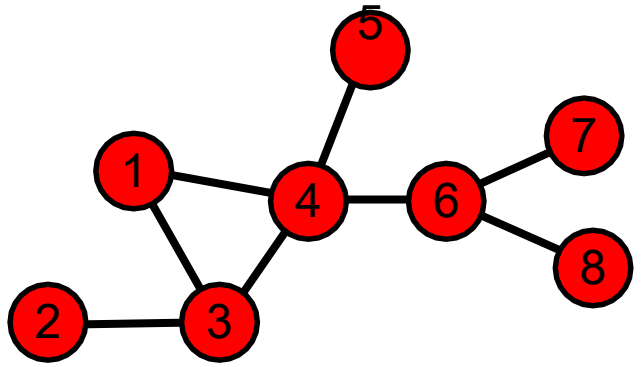
정점 u와 v의 사이의 경로(Path)는 아래 조건을 만족하는 정점들의 순열(Sequence)입니다

(1) u에서 시작해서 v에서 끝나야 합니다
(2) 순열에서 연속된 정점은 간선으로 연결되어 있어야 합니다

경로가 아닌 순열의 예시:

1, 6, 8

1, 3, 4, 5, 6, 8



© NAVER Connect Foundation

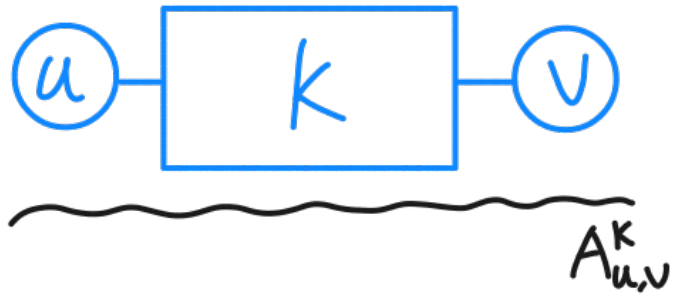
3.2 경로 기반 접근법

경로 기반 접근법에서는 두 정점 사이의 경로가 많을 수록 유사하다고 간주합니다

두 정점 u와 v의 사이의 경로 중 거리가 k인 것은 수는 $\mathbf{A}^k_{u,v}$ 와 같습니다 (직접 확인 결과, 진짜임)
즉, 인접 행렬 $\mathbf{A}$의 k 제곱의 u행 v열 원소와 같습니다

경로 기반 접근법의 손실 함수는 다음과 같습니다

$$\mathcal{L} = \sum_{(u,v)\in V\times V} \|\mathbf{z}_u^\top \mathbf{z}_v - \mathbf{A}^k_{u,v}\|^2$$

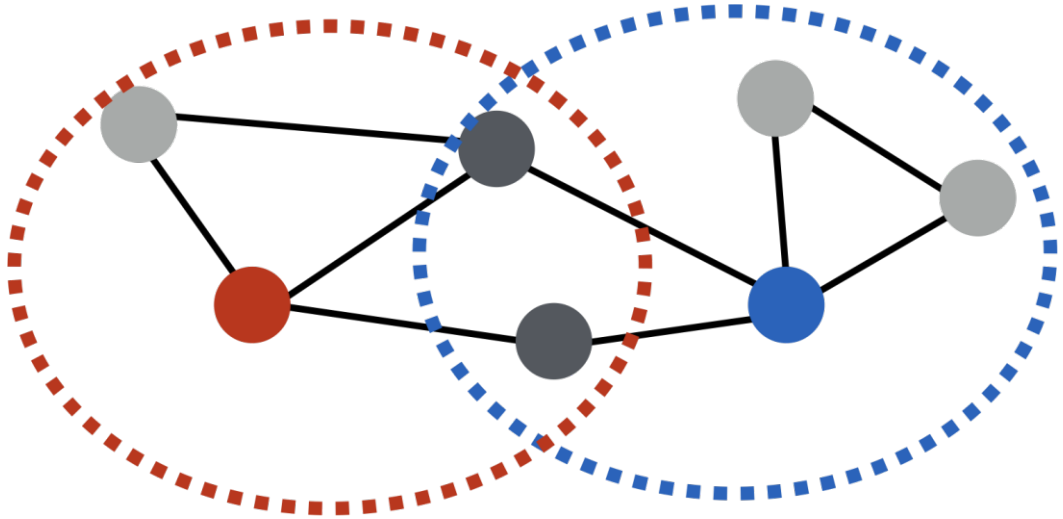


© NAVER Connect Foundation

3.3 중첩 기반 접근법

중첩 기반 접근법에서는 **두 정점이 많은 이웃을 공유**할 수록 유사하다고 간주합니다

아래 그림에서 빨간색 정점은 파란색 정점과 두 명의 이웃을 공유하기 때문에
유사도는 2가 됩니다



© NAVER Connect Foundation

이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

3.3 중첩 기반 접근법

중첩 기반 접근법에서는 두 정점이 많은 이웃을 공유할 수록 유사하다고 간주합니다

정점 u의 이웃 집합을 $N(u)$ 그리고 정점 v의 이웃 집합을 $N(v)$라고 하면
두 정점의 공통 이웃 수 $S_{u,v}$는 다음과 같이 정의 됩니다

$$S_{u,v} = |N(u) \cap N(v)| = \sum_{w \in N(u) \cap N(v)} 1$$

중첩 기반 접근법의 손실 함수는 다음과 같습니다

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \| \mathbf{z}_u^\top \mathbf{z}_v - \mathbf{S}_{u,v} \|^2$$

© NAVER Connect Foundation

3.3 중첩 기반 접근법

공통 이웃 수를 대신 자카드 유사도 혹은 Adamic Adar 점수를 사용할 수도 있습니다

자카드 유사도(Jaccard Similarity)는 공통 이웃의 수 대신 비율을 계산하는 방식입니다

$$\frac{|N_u \cap N_v|}{|N_u \cup N_v|}$$

Adamic Adar 점수는 공통 이웃 각각에 가중치를 부여하여 가중합을 계산하는 방식입니다

$$\sum_{w \in N_u \cap N_v} \frac{1}{d_w}$$

d_w: 공통 이웃 w의 연결성(degree)

-> 공통 이웃의 연결성(d_w)이 클 수록 의미가 퇴색됨
(예를 들어, 공통된 유명 아이돌을 팔로우 하는 두 사람에게 유사성을 기대하기 어렵다.)

© NAVER Connect Foundation

4. 임의보행 기반 접근법

그래프에서, 두 정점의 유사도 정의 방법 - 3

1. 임의보행 기반 접근법
2. DeepWalk와 Node2Vec
3. 손실 함수 근사

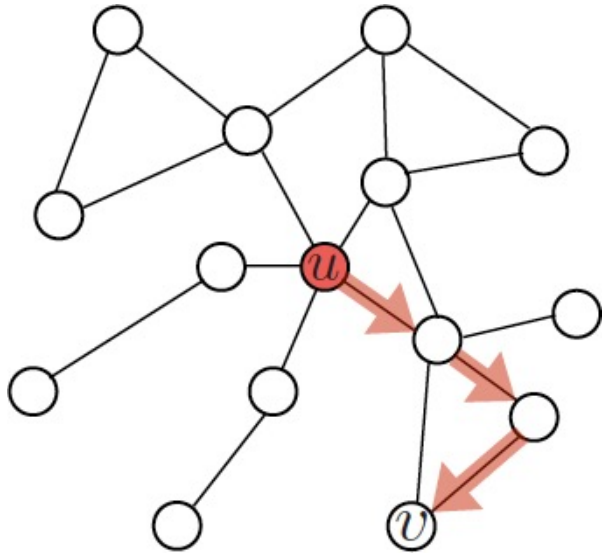
© NAVER Connect Foundation

4.1 임의보행 기반 접근법

임의보행 기반 접근법에서는 한 정점에서 시작하여 임의보행을 할 때 다른 정점에 도달할 확률을 유사도로 간주합니다

임의보행이란 현재 정점의 이웃 중 하나를 균일한 확률로 선택하는 이동하는 과정을 반복하는 것을 의미합니다

임의보행을 사용할 경우 시작 정점 주변의 지역적 정보와 그래프 전역 정보를 모두 고려한다는 장점이 있습니다



© NAVER Connect Foundation

이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

4.1 임의보행 기반 접근법

임의보행 기반 접근법은 세 단계를 거칩니다

1) 각 정점에서 시작하여 임의보행을 반복 수행합니다

2) 각 정점에서 시작한 임의보행 중 도달한 정점들의 리스트를 구성합니다
이 때, 정점 u에서 시작한 임의보행 중 도달한 정점들의 리스트를 $N_R(u)$라고 합시다
한 정점을 여러 번 도달한 경우, 해당 정점은 $N_R(u)$에 여러 번 포함될 수 있습니다

3) 다음 손실함수를 최소화하는 임베딩을 학습합니다

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v|\mathbf{z}_u))$$

u에서 시작한 임의보행이 v에 도달할 확률을
임베딩으로부터 추정한 결과를 의미합니다

© NAVER Connect Foundation

4.1 임의보행 기반 접근법

어떻게 임베딩으로부터 도달 확률을 추정할까요?

정점 u에서 시작한 임의보행이 정점 v에 도달할 확률 $P(v|z_u)$을 다음과 같이 추정합니다

$$P(v|\mathbf{z}_u) = \frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^\top \mathbf{z}_n)}$$

즉 유사도 $\mathbf{z}_v^\top \mathbf{z}_u$가 높을 수록 도달 확률이 높습니다

© NAVER Connect Foundation

4.1 임의보행 기반 접근법

추정한 도달 확률을 사용하여 손실함수를 완성하고 이를 최소화하는 임베딩을 학습합니다

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log \left(\frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^\top \mathbf{z}_n)} \right)$$

모든 시작점에 대하여 합산

임의보행 중 마주친 모든 정점에 대하여 합산

임베딩으로부터 추정한 도달 확률

© NAVER Connect Foundation

4.2 DeepWalk와 Node2Vec

임의보행의 방법에 따라 DeepWalk와 Node2Vec이 구분됩니다

DeepWalk는 앞서 설명한 기본적인 임의보행을 사용합니다
즉, 현재 정점의 이웃 중 하나를 균일한 확률로 선택하는 이동하는 과정을 반복합니다

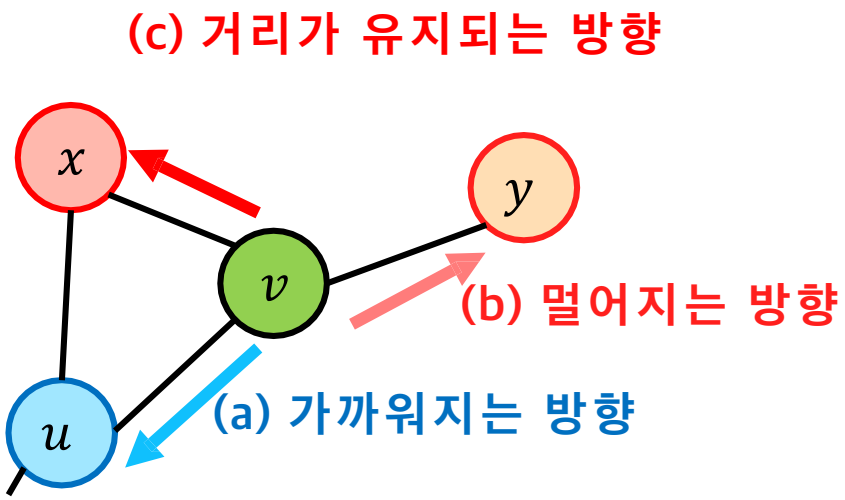
© NAVER Connect Foundation

4.2 DeepWalk와 Node2Vec

Node2Vec은 2차 치우친 임의보행(Second-order Biased Random Walk)을 사용합니다

현재 정점(예시에서 v)과 직전에 머물렀던 정점(예시에서 u)
을 모두 고려하여 다음 정점을 선택합니다

직전 정점의 거리를 기준으로 경우를 구분하여 차등적인 확률을 부여합니다



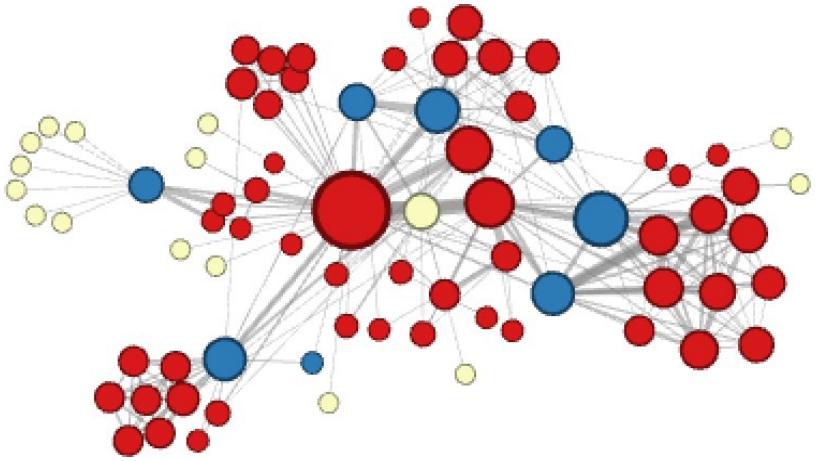
© NAVER Connect Foundation

이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

4.2 DeepWalk와 Node2Vec

Node2Vec에서는 부여하는 확률에 따라서 다른 종류의 임베딩을 얻습니다

아래 그림은 Node2Vec으로 임베딩을 수행한 뒤, K-means 군집 분석을 수행한 결과입니다



멀어지는 방향에 높은 확률을 부여한 경우



가까워지는 방향에 높은 확률을 부여한 경우

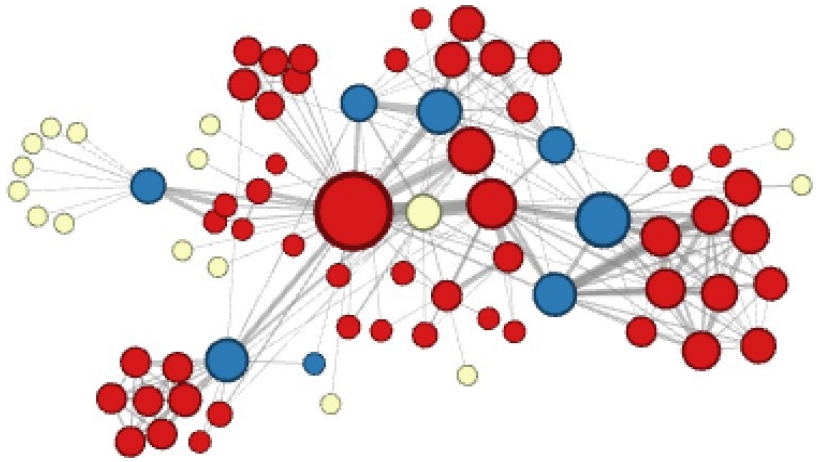
© NAVER Connect Foundation

이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

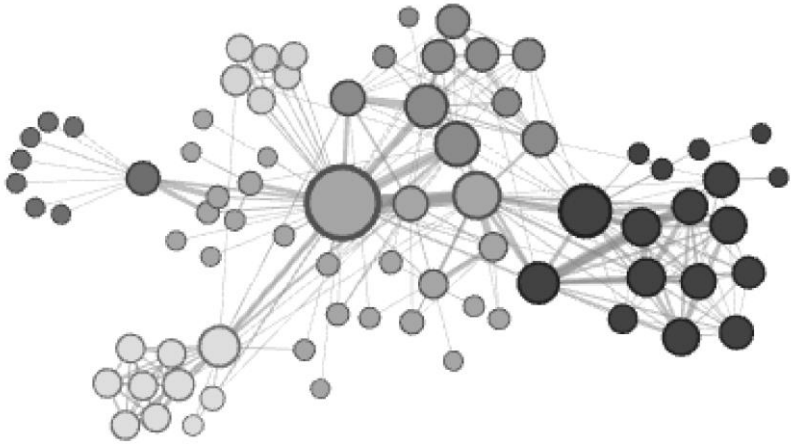
4.2 DeepWalk와 Node2Vec

Node2Vec에서는 부여하는 확률에 따라서 다른 종류의 임베딩을 얻습니다

멀어지는 방향에 높은 확률을 부여한 경우,
정점의 역할(다리 역할, 변두리 정점 등)이 같은 경우 임베딩이 유사합니다



멀어지는 방향에 높은 확률을 부여한 경우



가까워지는 방향에 높은 확률을 부여한 경우

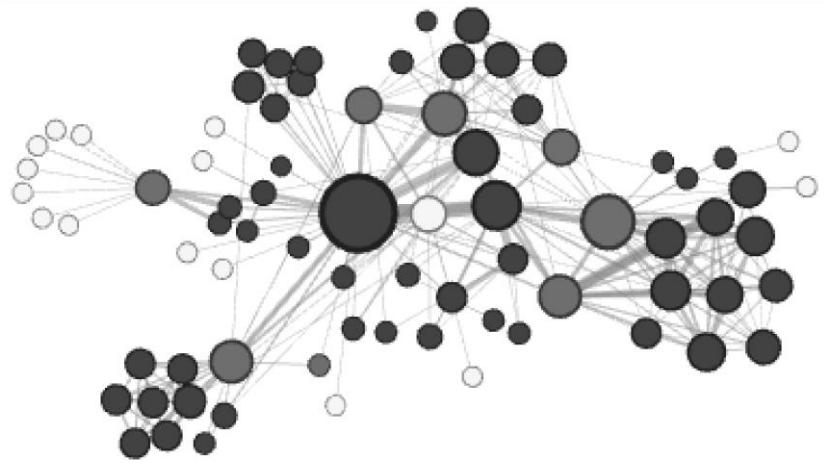
이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

© NAVER Connect Foundation

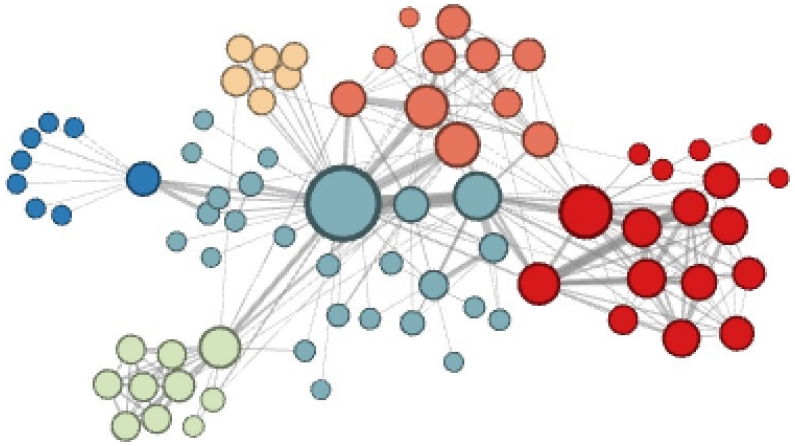
4.2 DeepWalk와 Node2Vec

Node2Vec에서는 부여하는 확률에 따라서 다른 종류의 임베딩을 얻습니다

가까워지는 방향에 높은 확률을 부여한 경우,
같은 군집(Community)에 속한 경우 임베딩이 유사합니다



멀어지는 방향에 높은 확률을 부여한 경우



가까워지는 방향에 높은 확률을 부여한 경우

© NAVER Connect Foundation

이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

4.3 손실 함수 근사

임의보행 기법의 손실함수는 계산에 정점의 수의 제곱에 비례하는 시간이 소요됩니다

중첩된 합 때문입니다

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log\left(\frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^\top \mathbf{z}_n)}\right)$$

정점이 많은 경우, 제곱은 매우 큰 숫자입니다
예시로 1억의 제곱은 1경 = 10,000조 입니다

© NAVER Connect Foundation

4.3 손실 함수 근사

따라서 많은 경우 근사식을 사용합니다

모든 정점에 대해서 정규화하는 대신 몇 개의 정점을 뽑아서 비교하는 형태입니다
이 때 뽑힌 정점들을 네거티브 샘플이라고 부릅니다

$$\log\left(\frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n\in V}\exp(\mathbf{z}_u^\top \mathbf{z}_n)}\right)$$

$$\approx \log(\sigma(\mathbf{z}_u^\top \mathbf{z}_v)) - \sum_{i=1}^{k}\log(\sigma(\mathbf{z}_u^\top \mathbf{z}_{n_i})), n_i \sim P_V$$

sigmoid 함수

확률 분포

© NAVER Connect Foundation

4.3 손실 함수 근사

따라서 많은 경우 근사식을 사용합니다

연결성에 비례하는 확률로 네거티브 샘플을 뽑으며,
네거티브 샘플이 많을 수록 학습이 더욱 안정적입니다

$$\log\left(\frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n\in V}\exp(\mathbf{z}_u^\top \mathbf{z}_n)}\right)$$

$$\approx \log(\sigma(\mathbf{z}_u^\top \mathbf{z}_v)) - \sum_{i=1}^{k}\log(\sigma(\mathbf{z}_u^\top \mathbf{z}_{n_i})), n_i \sim P_V$$

sigmoid 함수

확률 분포

© NAVER Connect Foundation

5. 변환식 정점 표현 학습의 한계

1. 변환식 정점 표현 학습과 귀납식 정점 표현 학습
2. 변환식 정점 표현 학습의 한계

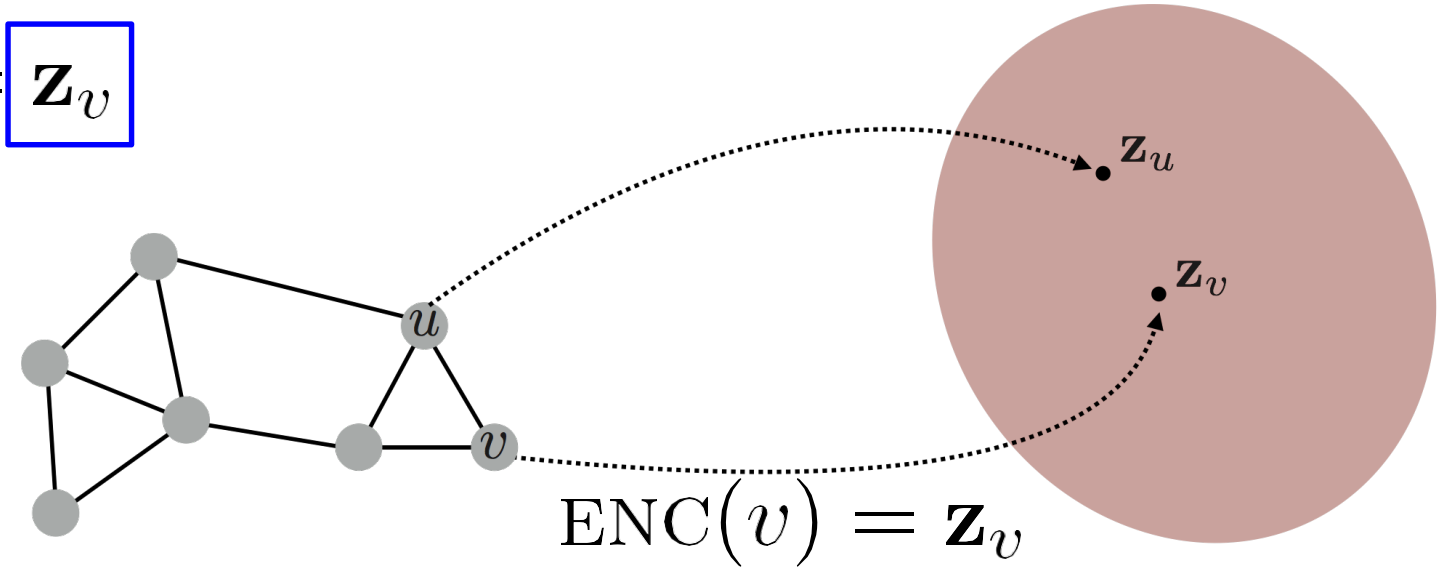
© NAVER Connect Foundation

5.1 변환식 정점 표현 학습과 귀납식 정점 표현 학습

지금까지 소개한 정점 임베딩 방법들을 변환식(Transductive) 방법입니다

변환식(Transdctive) 방법은 학습의 결과로 정점의 임베딩 자체를 얻는다는 특성이 있습니다
정점을 임베딩으로 변화시키는 함수, 즉 인코더를 얻는 귀납식(Inductive) 방법과 대조됩니다

$$\text{ENC}(v) = \mathbf{z}_v$$



© NAVER Connect Foundation

이미지 출처: http://snap.stanford.edu/proj/embeddings-www/

5.2 변환식 정점 표현 학습의 한계

변환식 임베딩 방법은 여러 한계를 갖습니다

1) 학습이 진행된 이후에 추가된 정점에 대해서는 임베딩을 얻을 수 없습니다

2) 모든 정점에 대한 임베딩을 미리 계산하여 저장해두어야 합니다

3) 정점이 속성(Attribute) 정보를 가진 경우에 이를 활용할 수 없습니다

9강에서는 이런 단점을 극복한 귀납식 임베딩 방법을 소개할 예정입니다
대표적인 귀납식 임베딩 방법이 바로 그래프 신경망(Graph Neural Network)입니다

© NAVER Connect Foundation